HoloNet Retrospective & Memory Neuron Architecture

HoloNet Retrospective & Memory Neuron Architecture

June 2026 — from nine experiments through four abstraction layers to one unified design.

---

1. Executive Summary

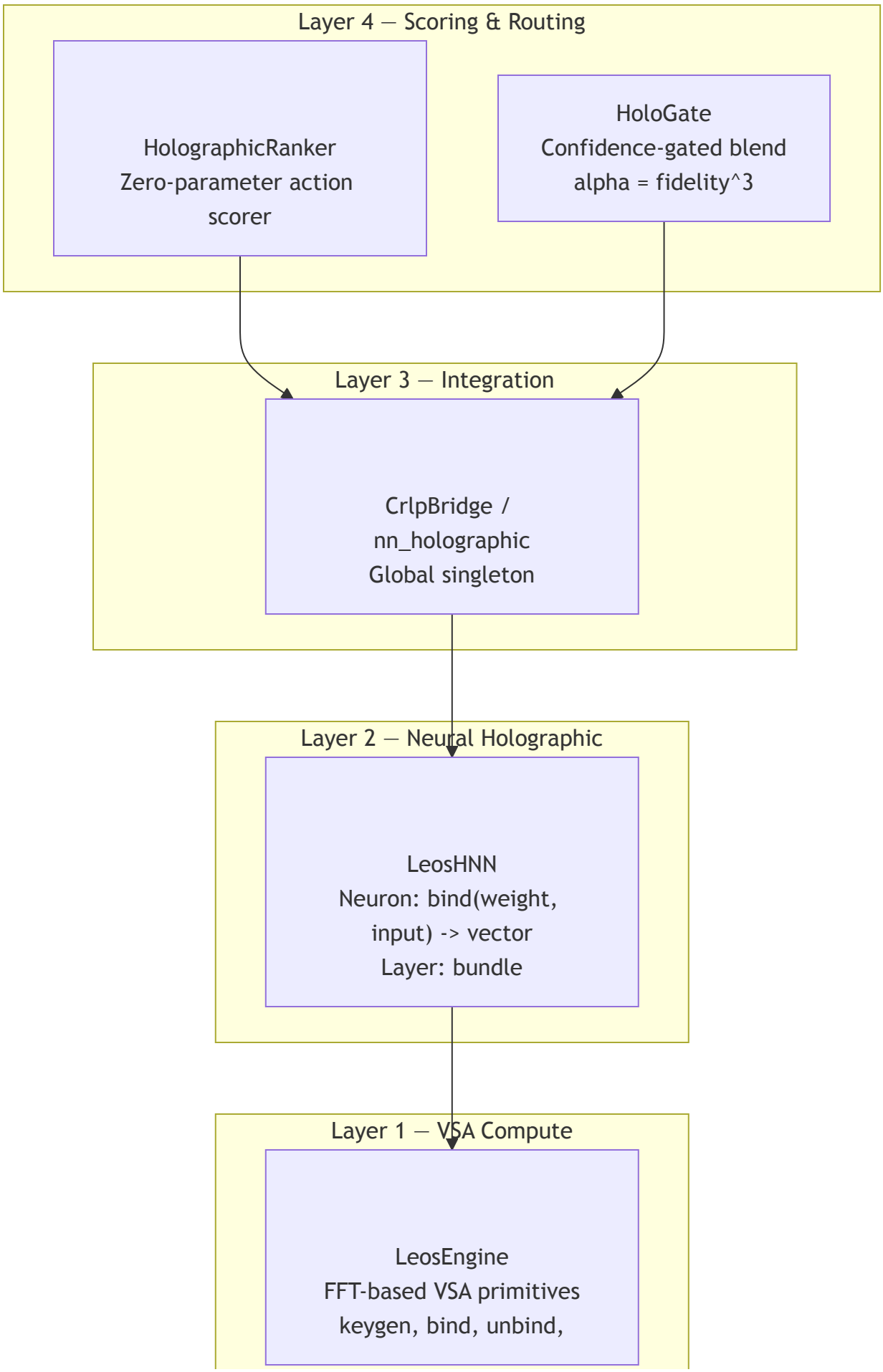
We ran nine experiments across four layers of abstraction, all pointing toward the same conclusion: **holographic memory shouldn't replace neural networks — it should augment them on a slower clock.**

The winning pattern is a memory neuron: a holographic trace that lives next to a traditional neuron, accumulates verified experience on a slow tick, and feeds context features into the neural forward pass on every fast tick. The neural net learns through normal SGD whether and how to use those features. No post-hoc blending, no separate architecture, no confidence gate.

This document captures every experiment finding, the architectural proposal it supports, and exactly what code we keep, cut, or build.

---

2. The Four-Layer Stack


Layer 4 — Scoring & Routing
HolographicRanker
Zero-parameter action scorer
HoloGate
Confidence-gated blend
alpha = fidelity^3
Layer 3 — Integration
CrlpBridge / nn_holographic
Global singleton
Layer 2 — Neural Holographic
LeosHNN
Neuron: bind(weight, input) -> vector
Layer: bundle
Layer 1 — VSA Compute
LeosEngine
FFT-based VSA primitives
keygen, bind, unbind,

Layer 1 is the compute substrate — it stays. **Layer 2** is a full VSA-space neural architecture that we deprecate because k-NN is the proven core and layer training doesn't converge. **Layer 3** gets folded into the memory neuron grid. **Layer 4** gets removed: HolographicRanker becomes the computation *inside* each memory neuron, and HoloGate's post-hoc blending gets replaced by learned feature weighting in SGD.

---

3. Experiment Lineage

Nine experiments, each answering one question and feeding the next.



The **claim_reform** experiment was the turning point: six encoding strategies proved that raw continuous state fails because holographic memory needs states in the same decision region to map to the same key. Bucketed or gate-aware encoding works. This insight drove the entire memory neuron design.

---

4. What Each Experiment Taught

Experiment	Question	Finding
`holo_ranker_demo`	Can holo k-NN replace the MLP?	Yes — on the right tasks. 0 params, graceful forgetting at 1/sqrt(N).
`holo_all_experiments`	Where does holo win and lose?	Dominates spatial/local tasks; struggles on logical/conditional — but it's an encoding problem, not an architecture problem.
`holo_claim_reform`	Why does holo fail at delayed claim?	**The key finding.** Six strategies tested. Bucketed-by-decision-region works. Gate-aware encoding works best. Holo needs states in the same decision region to map to same key.
`holo_envelope`	Can 4 universal holo features improve domain-specific MLP?	Yes. Tiny 4->8->4->1 MLP trained on nothing but holo scores achieves competitive accuracy across domains.
`holo_replace_envelope`	Can we replace domain features entirely?	Yes. domain_state -> uint64_t seed -> keygen(seed) -> vector. Zero domain knowledge.
`hybrid_holo_nn`	Each neuron as its own tiny holographic store?	Works. Each neuron binds(input_fragment, output_component) then bundles. Routing is naive (hash-based), though.
`holonet`	Sparse activation with feature-space routing?	16 neurons, K=4 active, distance-weighted scoring. Natural specialization emerges. Right routing, but centroids are random.
`fully_holographic`	Entire network in VSA space?	Yes — memory recall -> layer -> classify. Graceful forgetting to 800 distractors. But it's a separate architecture, not integrated.
`gated_runner_demo` / `gated_all`	Blend holo + neural scores?	alpha = fidelity^3 auto-routes: pure holo at N=1, pure neural at N=100. But it's post-hoc glue — two independent systems averaged at the end.

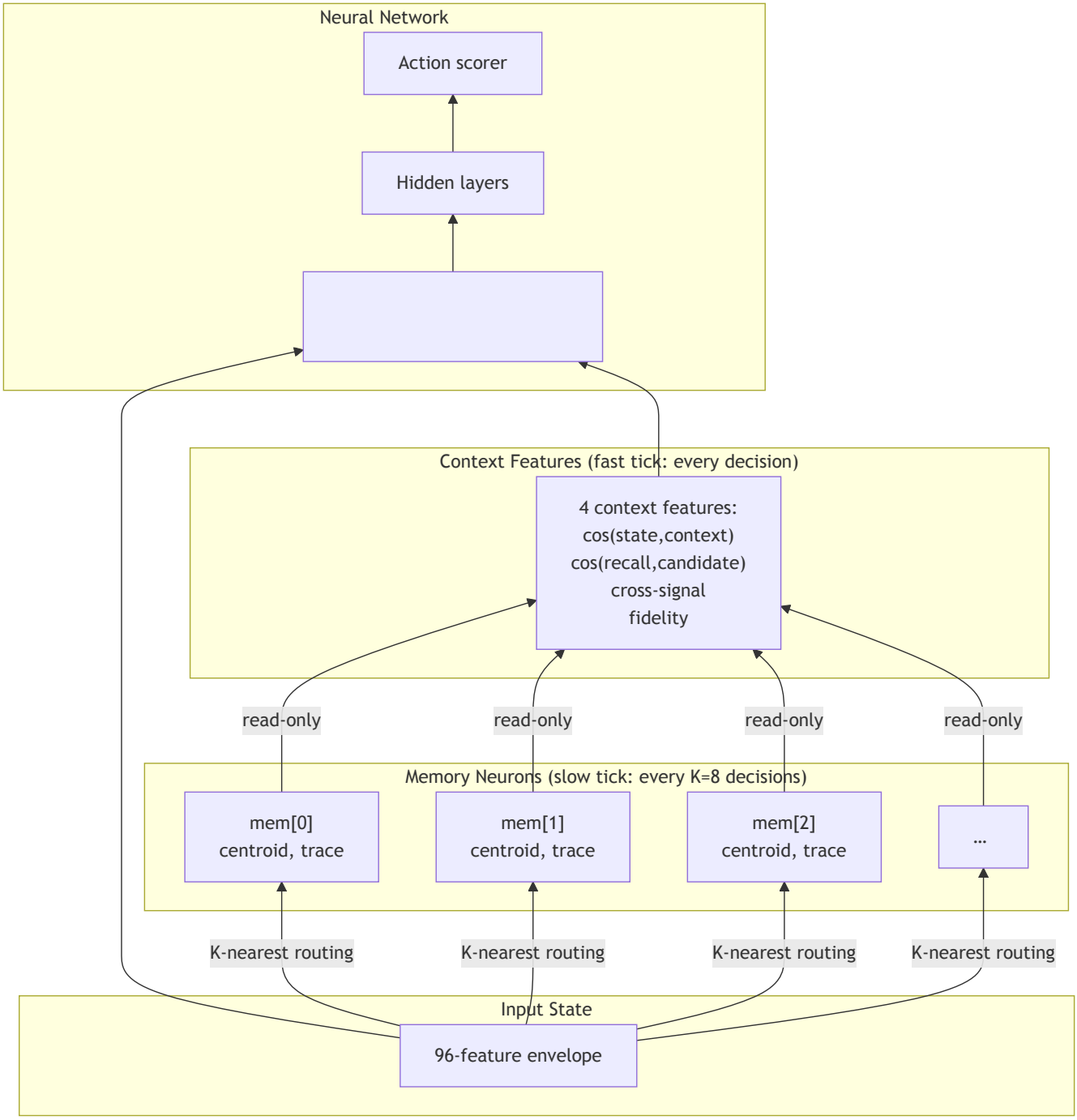
Experiment	Question	Finding
`holographic_prime_gap`	Holo features for prime gap prediction?	4 holo context + 14 static > 14 static alone. Cluster-based encoding is the right approach.
`cross_domain_big`	Shared memory, transfer, voids, scale?	Shared memory across 4 tasks works with modest interference. Voids are detectable. Scales to 10K frames at ~0 us/query.

---

5. The Core Insight

> **Post-hoc blending is the wrong architecture.** You shouldn't run two independent systems and average their scores. The memory should feed context into the neural network so SGD itself learns the weighting.

This directly implies the memory neuron design:



Fast tick (every decision): Memory neurons are read-only. They provide 4 context features appended to the 96-feature envelope. The neural net forward pass proceeds normally with these 100 features.

Slow tick (every K=8 decisions): After the verifier confirms a correct action, the K nearest memory neurons store `bind(state, verified_action)` into their holographic trace. Centroids drift toward stored states at lr=0.01. Context features are recomputed for the next fast tick.

---

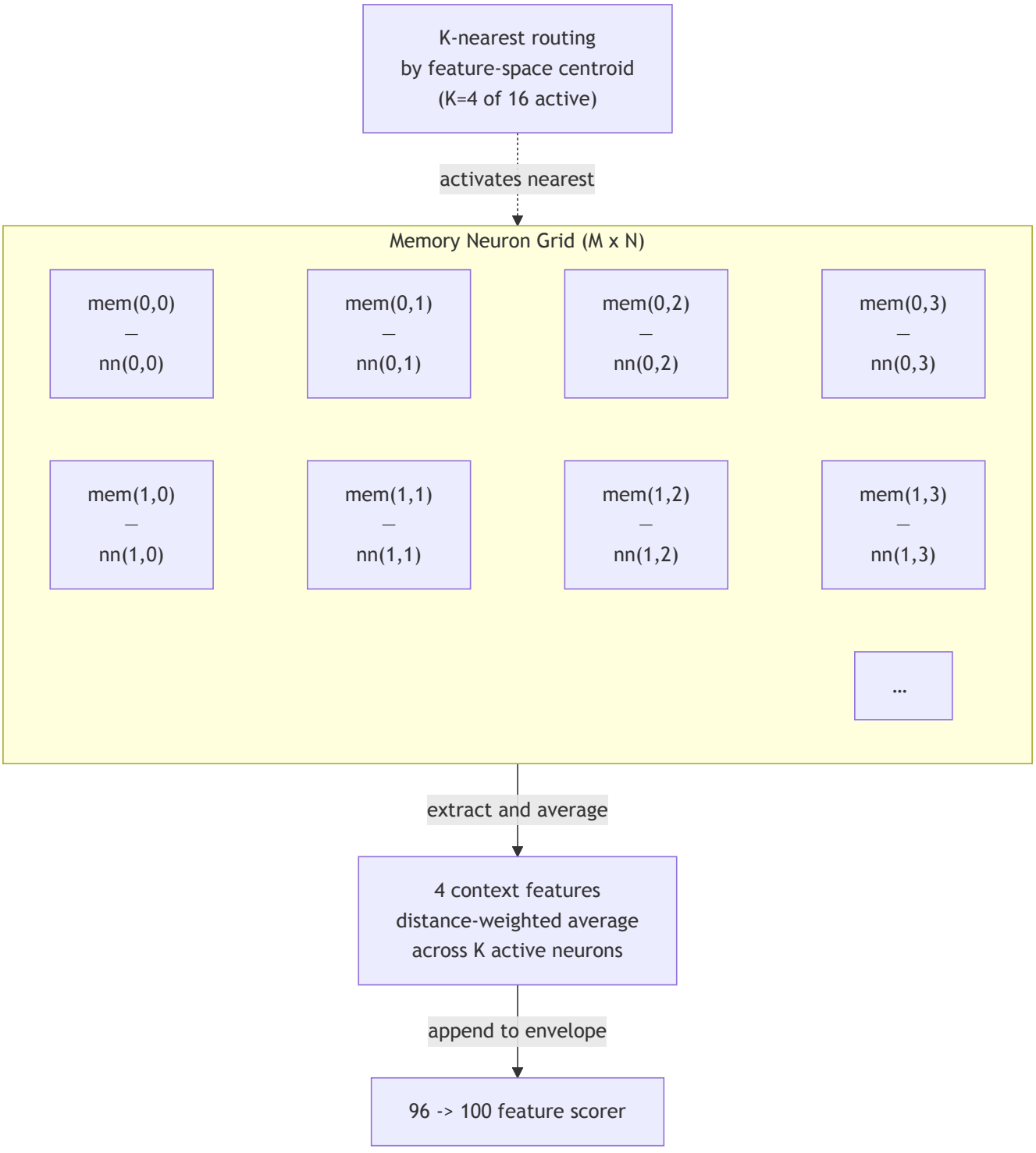
6. Proven Claims

These have been verified across multiple experiments:

1. **Holographic k-NN achieves 100% on local-continuity tasks** with zero parameters. Fidelity decays as 1/sqrt(N). Memory is O(dim) fixed.
2. **Flat-spectrum keygen produces perfect bind/unbind roundtrip.** `sim(v, unbind(bind(k,v), k)) = 1.000000` verified.
3. **Radix-2 FFT at ~10 GFLOPS, 3e-15 roundtrip error.** Self-contained, pre-computed twiddles, zero malloc in hot path.
4. **Memory fades gracefully — no catastrophic forgetting.** Accuracy holds at 100% from 5 to 500 stored examples despite fidelity dropping from 0.45 to 0.045.
5. **Neural scorer and holographic k-NN are complementary.** 84% vs 30.5% on delayed_claim (logical/discontinuous); 100% vs lower on grid_path (spatial/local).
6. **HoloNet with structured receptive fields matches flat k-NN** while distributing load across neurons through sparse activation.
7. **Zero-parameter architecture.** No weights to train, checkpoint, or serialize. The model IS the memory trace. Training IS storing examples.

---

7. The Memory Neuron Grid



Optimal Default Configuration

Parameter	Value	Reason
Grid dimensions (MxN)	4x4 = 16 neurons	Matches holonet.c; enough for specialization, small enough for

Parameter	Value	Reason
		sparse activation
Active neurons (K)	4	25% activation; distance-weighted scoring works at this ratio
VSA dimension	2048	Sweet spot from all experiments; 16KB per trace, good fidelity
Slow tick interval	8	Fast enough to accumulate signal, slow enough for stable context
Context features	4	Proven sufficient by holo_envelope.c
Centroid learning rate	0.01	Gentle online clustering
Total memory	16 x 16KB = 256KB	Negligible

---

8. The Four Context Features

From the CrlpBridge experiments, these four signals carry all the necessary memory context:

#	Feature	Formula	Meaning
1	State familiarity	`cos(state, holo_context(state))`	Has this region been seen before?
2	Candidate match	`cos(candidate, holo_recall_recent(state))`	Does this candidate match memory?
3	Cross-signal	`cos(holo_context(state), candidate)`	Alignment between context and candidate
4	Fidelity	`1 / sqrt(stored)`	How confident is this memory?

These are extracted per active memory neuron and distance-weighted-averaged across the K active ones before being appended to the scorer envelope. Since they're just 4 additional input features, the neural net naturally learns to weight them — no post-hoc blending, no confidence gate, no separate path.

---

9. Keep, Cut, Build

Keep

Artifact	Why
LeosEngine	Clean, fast, correct VSA compute substrate. The foundation.
HolographicRanker	Becomes the core computation inside each MemoryNeuron.
memory_neuron.h	Already written. The header defines the new abstraction.

Cut

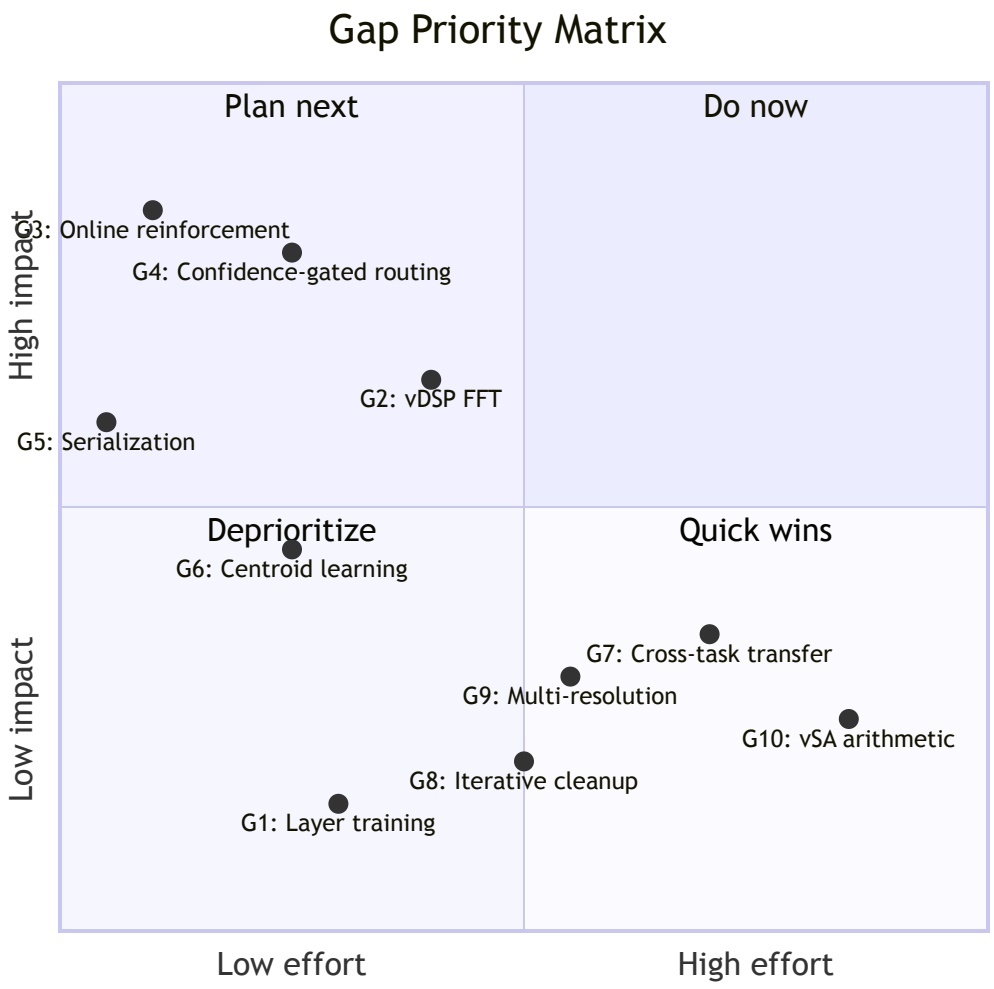
Artifact	Reason
LeosHNN / fully_holographic.c	Separate VSA neural network — wrong direction. Memory should augment, not replace.
HoloGate / gated_all.c / gated_runner_demo.c	Post-hoc blending replaced by learned feature weighting in SGD.
HoloRunner, HoloExperiment, HoloGate	Experiment harnesses replaced by unified MemoryNeuron grid.
CrlpBridge / nn_holographic	Global singleton folded into memory neuron grid.
holonet.c, hybrid_holo_nn.c, holo_envelope.c, holo_replace_envelope.c, holo_claim_reform.c, holo_all_experiments.c	Exploration code; learnings captured in this document.
holographic_prime_gap.c, holographic_ranker_demo.c, cross_domain_big.c	Archive or rewrite against MemoryNeuron grid.
scripts/merge_holo.py	Already untracked.

Build

Artifact	Effort	What it does
`memory_neuron.c`	~2 days	Implements the grid: create, destroy, context extraction, verified storage, centroid drift
Modified `nn_scorer_envelope_fill_eml()`	~1 hour	Accepts 4 optional holo context features (96 -> 100)
Wiring into eval/train loop	~1 hour	holo_ranker_store(state, correct_action) after verifier confirmation

---

10. Gaps & Priority

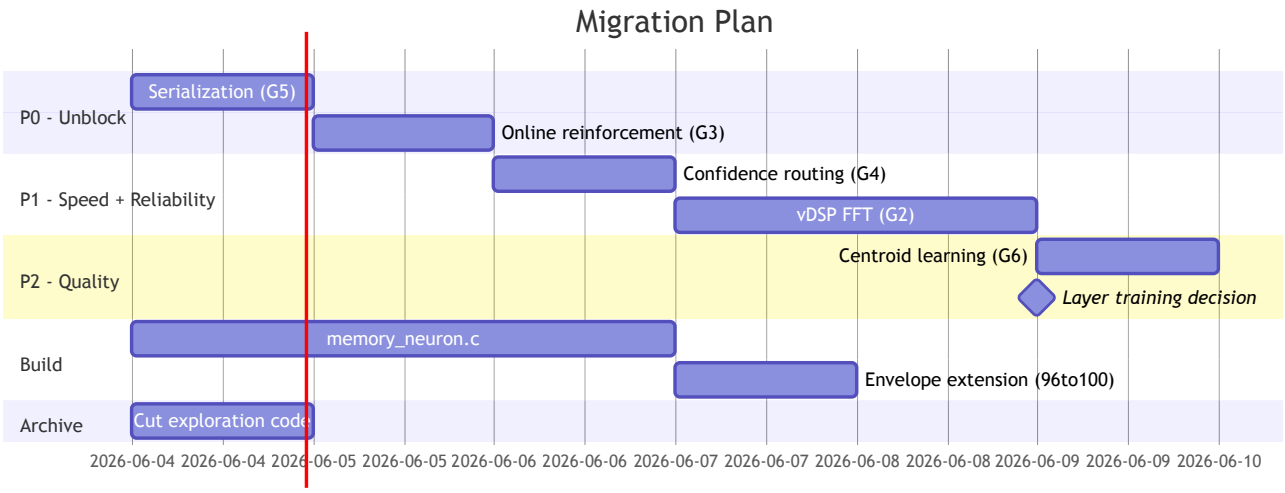


Priority	Gap	Effort	Impact
P0	**G3: Wire online reinforcement** — holo_ranker_store after verifier confirm	1h	Makes ranker usable in real pipelines
P0	**G5: Serialization** — fwrite(M, dim*sizeof(double), fp)	30m	Checkpointing and model transfer
P1	**G4: Confidence-gated routing** — blend alpha = fidelity; fall back to neural when fidelity < 0.1	1d	Combined holo+neural wins on all tasks
P1	**G2: vDSP FFT** — replace hand-rolled FFT with Accelerate/FFTW	2d	3-5x throughput improvement

Priority	Gap	Effort	Impact
P2	**G6: HoloNet centroid learning** — online k-means instead of random init	1d	Better neuron utilization (currently 8/16 unused)
P2	**G1: Accept k-NN as core** — deprecate layer training, ship what works	0	Removes dead-end research path
P3	**G7: Cross-task transfer** — Rosetta projection or task-tagged keys	3d	Knowledge sharing between domains
P3	**G9: Multi-resolution memory** — pyramid of traces at dim/2, dim/4, dim/8	2d	Coarse-to-fine retrieval
P4	**G8: Iterative cleanup** — non-normalized intermediate accumulator	2d	Recover deeply buried memories
P4	**G10: vSA arithmetic** — RNS encoding for exact integer ops in VSA space	5d	Future research direction

---

11. Migration Roadmap



1. **Cut first.** Delete or archive all the exploration code listed in the cut table. This reduces the surface area by ~12 files and prevents confusion about which architecture is canonical.
2. **Unblock.** Serialization (30 minutes) and online reinforcement (1 hour) are trivial to implement and immediately make the ranker usable.
3. **Build.** Implement `memory_neuron.c` against the header that already exists. Wire it into the eval/train loop. Extend the scorer envelope.
4. **Accelerate.** vDSP FFT and confidence-gated routing together give both speed and robustness.
5. **Refine.** Centroid learning makes the sparse activation actually work (instead of leaving half the neurons idle with random centroids).
6. **Explore.** Cross-task transfer and multi-resolution memory are high-effort research directions; pursue only when the core architecture is stable.

---

12. Design Principles (Learned)

1. **Memory should augment, not replace.** The holographic store is good at local-continuity recall; the neural net is good at discontinuous logical mappings. The right architecture feeds one into the other, not one instead of the other.
2. **Encoding matters more than architecture.** The claim_reform experiment proved that raw continuous state vectors don't work for holographic memory because near-orthogonality in high dimensions means every state looks different. Bucketing states by decision region fixes this.
3. **SGD learns weighting better than hand-tuned blending.** The confidence gate (alpha = fidelity^3) works, but it's a ceiling: two independent systems averaged at the end. Giving SGD those same signals as input features lets it discover more nuanced weightings.
4. **Slow memory, fast network.** Memory updates on a slower clock (every K decisions) while the neural forward pass runs on every decision. This matches the biological pattern: memory consolidation is slower than perception/action.
5. **Sparse activation with learned centroids beats dense or random.** The HoloNet experiment showed natural specialization when routing is distance-weighted, but random centroids leave half the neurons unused.
6. **Zero parameters is a feature, not a limitation.** Training IS storing. The model IS the trace. Serialization is 2 lines of fwrite. This is a fundamentally different — and simpler — paradigm than SGD-trained weights.